



Programmieren

Falk Morgenstern

Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

5. Vererbung

5.6 Prinzipien der Objektorientierung



Ansätze zum Objektorientierten Design

- Nach Einführung von Objekten, Klassen, Vererbung und Interfaces in Java
 - Begriffe zur Objektorientierung (Quelle: Handbuch der Java-Programmierung)
 - Design Prinzipien Objektorientierten Designs (Quelle: Head First, Object-Oriented Analysis and Design)
- Ansätze, die generell in objektorientierten Sprachen möglich sind.



Begriffe zur Objektorientierung (Wiederholung)

- **Abstraktion**
 - Eine Klasse abstrahiert die Eigenschaften konkreter Objekte
- **Kapselung**
 - Trennung von interner Implementierung und äußerer Schnittstelle
 - Die Komplexität des Aufbaus des Objekts ist in der Klasse gekapselt
- **Klassenbeziehungen**
 - is-a Beziehung (Vererbung; Generalisierung, Spezialisierung)
 - part-of Beziehung (Aggregation, Komposition)
 - use Beziehung (Verwendungs- oder Aufrufbeziehung)
- **Polymorphie** / dynamische Bindung
 - Bei Vererbung und überschriebenen Methoden wird die ausgeführte Methode zur Laufzeit bestimmt (dynamische Bindung)



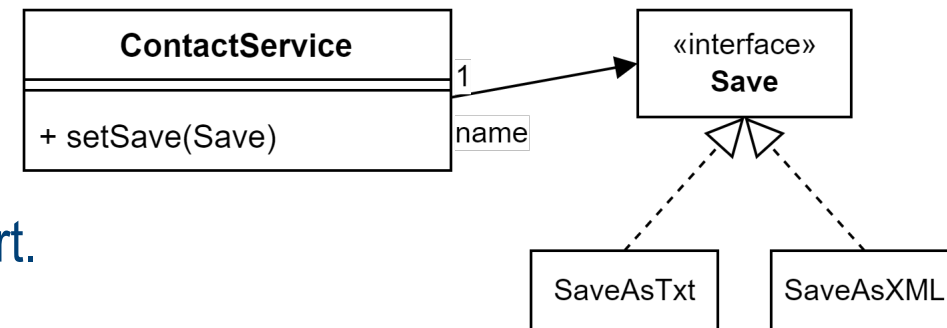
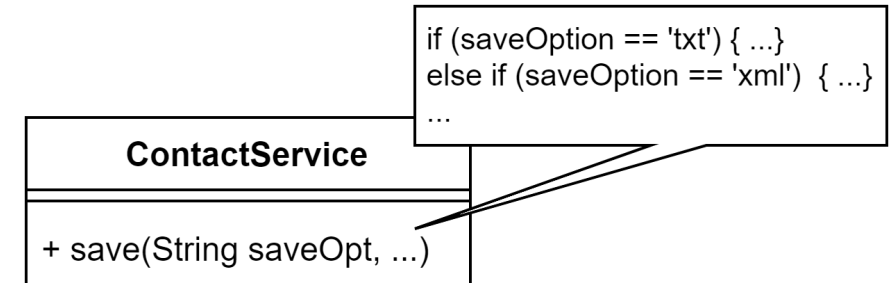
Design-Prinzipien für den objektorientierten Entwurf

- Unterstützen bei der Frage, wie man sinnvoll eine Klassenstruktur erstellt
- Generell
 - Implementierungsdetails von Objekten werden in einer Klasse gekapselt. (Encapsulate what varies.)
 - Programmieren Sie auf eine Schnittstelle, nicht auf eine Implementierung. (Code to an interface rather than to an implementation.)
- Weitere Design-Prinzipien, die im folgenden vorgestellt werden
 - Open-Closed Principle (OCP)
 - Don't repeat yourself (DRY)
 - The Single Responsibility Principle (SRP)
 - Liskov Substitution Principle (LSP)



Open-Close Principle (OCP)

- Klassen sollten offen für Erweiterungen, aber geschlossen für Änderungen sein (sinngemäß nach Bertrand Meyer, Object Oriented Software Construction).
- Anti-Beispiel:
 - Das Prinzip wird verletzt von einer Klasse `ContactService`, die verschiedene Speicheroptionen anbietet, auswählbar über den ersten Aufrufparameter. Soll eine weitere Option hinzugefügt werden, muss die Klasse ergänzt werden.
- Verbesserung:
 - Interfaces nutzen; Vererbung und Überschreiben von Methoden
 - Hier: `ContactService` soll unverändert bleiben; Speicheroptionen werden in eigenen Klassen implementiert. Dazu wird das Interface `Save` eingeführt:





Open-Close Principle (OCP): Beispiel

```
public class ContactService {  
    private Contact contact;  
    private Save save;  
  
    public void setSave(Save save) { this.save = save; }  
  
    public void save(String filename) { save.save(contact, filename); }  
}
```

```
public interface Save {  
    void save(Contact contact, String filename);  
}
```

```
public class SaveAsTxt implements Save {  
    @Override  
    public void save(Contact contact, String filename) { /*...*/ }  
}
```

```
public class SaveAsXML implements Save {  
    @Override  
    public void save(Contact contact, String filename) { /*...*/ }  
}
```



Don't repeat yourself (DRY)

- Vermeiden Sie, Code zu kopieren! Extrahieren Sie Code-Fragmente, die Sie mehrfach an verschiedenen Stellen nutzen wollen, und schaffen Sie eine Implementierung an einer einzigen Stelle.
- Beispiele:
 - Faktorisieren Sie gleiche Code-Fragmente heraus, indem Sie eine Methode verwenden.
 - Überlegen Sie gut, zu welcher Klasse diese Methode gehört.
 - Manchmal werden Sie eine neue Klasse entwerfen, zu der die Methode gehört.
- Frage: Bei der Klasse Animal haben wir in jeder Unterklasse das Verhalten jeweils neu implementiert. Wie kann man dabei Übereinstimmungen nutzen, z.B. bei der Implementierung der Klassen Cow, Sheep, Horse, die alle Gras fressen?



The Single Responsibility Principle (SRP)

- Jede Klasse sollte nur eine Zuständigkeit haben. Der Fokus sollte auf dieser einen Zuständigkeit liegen.
- Anders ausgedrückt: “Es sollte nie mehr als einen Grund dafür geben, eine Klasse zu ändern.” (Robert C. Martin, Agile Software Development)
- Anti-Beispiel:
 - Das Prinzip wird verletzt durch eine Spiel-Klasse, die direkt Ausgaben auf der Konsole macht. Sie müssten die Klasse ändern, wenn Sie eine GUI statt der Ausgabe auf der Konsole verwenden wollen.
- Verbesserung:
 - Die Methode zur Ausgabe wird in eine eigene Klasse ausgelagert. Änderung der Ausgabe ohne Änderung der Spiel-Klasse.



Liskov Substitution Principle (LSP)

- Sei $q(x)$ eine beweisbare Eigenschaft von Objekten x des Typs S . Dann soll $q(y)$ für Objekte y des Typs T wahr sein, wenn T ein Untertyp von S ist. (Barbara Liskov)
- Ein Objekt einer Unterklasse muss an allen Stellen einsetzbar sein, an denen Objekte der Oberklasse verwendet werden können.
- Beispiele:
 - Dog als Unterklasse von `Animal` erfüllt das Prinzip.
 - Quadrat als Unterklasse von `Rechteck` verletzt das Prinzip. Warum?

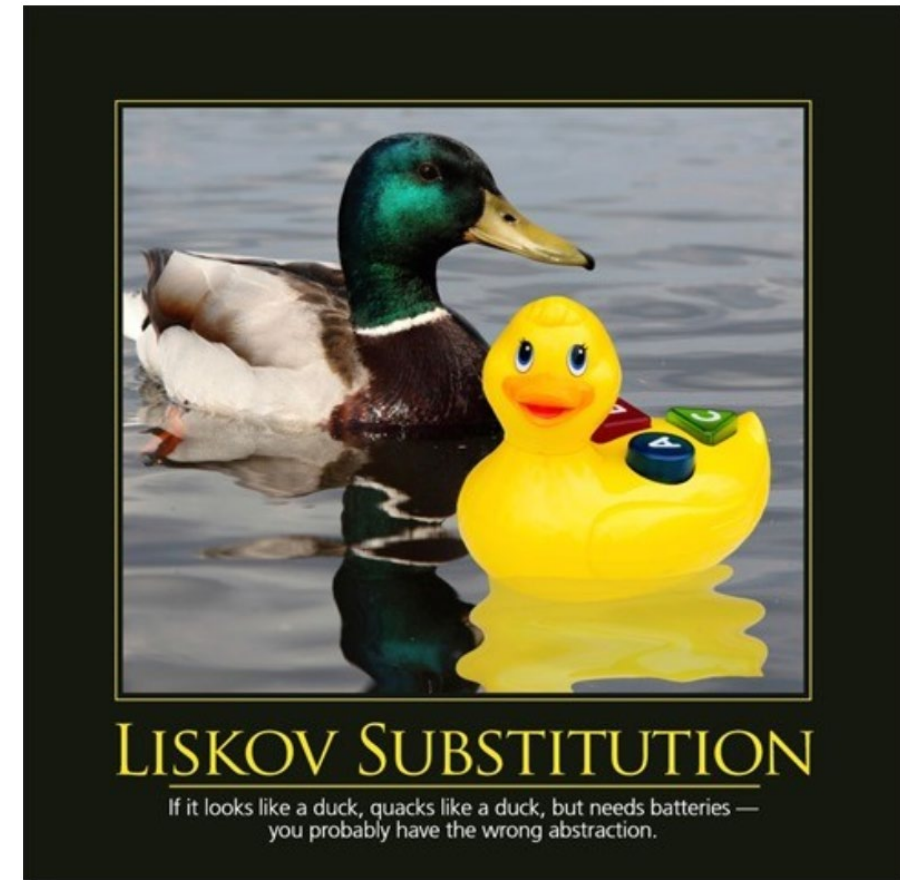


Liskov Substitution Principle (LSP)

- Ein Objekt einer Unterklasse muss an allen Stellen einsetzbar sein, an denen Objekte der Oberklasse verwendet werden können.
- Beispiele:
 - Dog als Unterklasse von Animal erfüllt das Prinzip.
 - Quadrat als Unterklasse von Rechteck verletzt das Prinzip.
 - Rechteck hat set-Methoden, um Höhe und Breite unabhängig zu setzen.
 - Also kann man beim Quadrat durch Vererbung auch Höhe und Breite unabhängig verändern.
 - Damit ist ein Quadrat nicht anstelle eines Rechtecks verwendbar, denn das LSP ist verletzt.
 - Verwenden Sie das LSP statt der is-a Beziehung.

Liskov Substitution Principle (LSP)

- LSP informell: Eine Unterklasse erweitert das Verhalten der Oberklasse um neue Aspekte, verändert das Verhalten der Oberklasse aber nicht. Daher kann ein Objekt der Unterklasse problemlos überall verwendet werden, wo ein Objekt der Oberklasse erwartet wird.





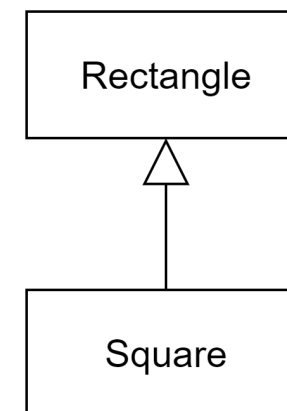
Beispiel: LSP verletzt

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public int getWidth() { return width; }  
  
    public void setWidth(int w) { width = w; }  
  
    public int getHeight() { return height; }  
  
    public void setHeight(int h) { height = h; }  
  
    public int getArea() { return width * height; }  
  
    public static void main(String[] args) {  
        Rectangle rect = new Square();  
        rect.setWidth(2);  
        rect.setHeight(5);  
        System.out.println(rect.getArea());  
    }  
}
```

// erwarte 10, Ausgabe ist aber 25
// verhält sich NICHT wie ein Rechteck!

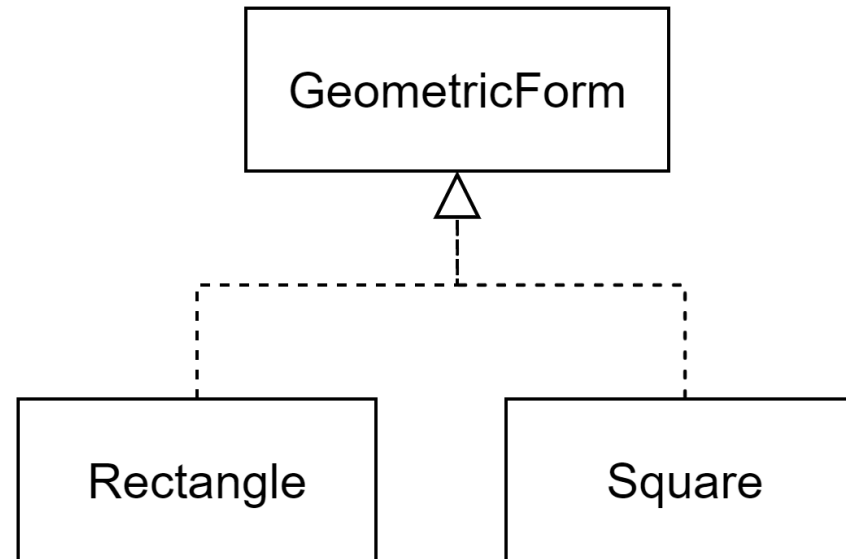


```
public class Square extends Rectangle {  
    @Override  
    public void setWidth(int w) {  
        super.setWidth(w);  
        super.setHeight(w);  
    }  
  
    @Override  
    public void setHeight(int h) {  
        super.setHeight(h);  
        super.setWidth(h);  
    }  
}
```





Beispiel: LSP verbessert





```
public class Oberklasse {  
    String z = 1;  
    protected Object o = b_meth("42");  
    abstract void a_meth();  
    private String b_meth(String s) { return s; }  
}
```



- Fehler finden

```
public class Unterklasse extends Oberklasse {  
    abstract void k();  
    void c_meth() {  
        b_meth("abc");  
        String s = o;  
        main(null);  
    }  
    public static void main(String[] args) {  
        System.out.println(o);  
    }  
}
```





```
1 public class A {
2     int a = 0;
3     public int m() { return a; }
4     public A(int x) { a = x; }
5     A() { }
6 }
7
8 public class B extends A {
9     int a = 0;
10    int b = 0;
11    public int m() {
12        return super.a;
13    }
14    public B(int a) {
15        this.a = a;
16    }
17
18    public static void main(String[] args) {
19        A a1 = new A(1);
20        A a2 = new B(2);
21        B b1 = new B(3);
22        System.out.println(a2 instanceof A); // Ausgabe:
23        System.out.println(a2 instanceof B); // Ausgabe:
24        System.out.println(b1 instanceof A); // Ausgabe:
25        System.out.println(a1.m());          // Ausgabe:
26        System.out.println(a2.m());          // Ausgabe:
27        System.out.println(b1.m());          // Ausgabe:
28        System.out.println(a2.a);            // Ausgabe:
29        System.out.println(b1.a);            // Ausgabe:
30    }
31 }
```



- a) Wie viele Instanzvariablen hat ein B-Objekt?
- b) Ohne Zeile 5 kompiliert das Programm nicht. Warum?
- c) Ergänzen Sie die Konsolen-Ausgaben direkt in den Zeilen 22 – 29.



```
public interface N {  
    int foo(double d);  
  
    default int bar(int x) {  
        return f();  
    }  
  
    private int f() {  
        return 12;  
    }  
}
```



```
public interface M {  
  
    abstract void a();  
  
    default int bar(int x) {  
        return 1;  
    }  
  
    static int h() { return 42; }  
}
```



- Ausgabe?

```
public class NImpl implements N, M {  
  
    public int foo(double d) { return 0; }  
  
    public int bar(int x) {  
        return N.super.bar(x) + M.super.bar(x);  
    }  
  
    public void a() { System.out.println("ok"); }  
  
    public static void main(String[] args) {  
        int x1 = M.h();  
        int x2 = new NImpl().bar(2);  
        int x3 = ((N)new NImpl()).foo(1.0);  
        System.out.println(x1 + " " + x2 + " " + x3); // Ausgabe?  
    }  
}
```

